
Complete Code Walkthrough

[Architecture](#) · [Data Models](#) · [Stage-by-Stage Guide](#) · [Bug Fixes](#) · [Deployment](#)

BI Manager's Technical Reference

Version 1.0 | June 2026

Table of Contents

1	Executive Summary
2	Architecture Overview
3	File Structure
4	Data Models — <code>utils/models.py</code>
5	Stage 1 — Parser Agent — <code>parsers/twb_parser.py</code>
6	Stage 2 — Power Query M Generator — <code>generators/mquery_generator.py</code>
7	Stage 3 — DAX Generator — <code>generators/dax_generator.py</code>
8	Stage 4 — Visual Generator — <code>generators/visual_generator.py</code>
9	Stage 5 — PBIX Assembler — <code>generators/pbix_assembler.py</code>
10	Stage 6 — Publisher — <code>generators/pbi_publisher.py</code>
11	Validator — <code>validators/coverage_validator.py</code>
12	Report Writer — <code>generators/report_writer.py</code>
13	Pipeline Orchestrator — <code>pipeline.py</code>
14	CLI — <code>cli.py</code>
15	Mapping Tables — <code>utils/mappings.py</code>
16	Bug Fixes — Code Review Findings
17	Enhancement Roadmap
18	End-to-End Example Walkthrough
19	Deployment Guide
20	Glossary

1. Executive Summary

The **Tableau to Power BI Migration Accelerator** is a standalone Python tool that automates the conversion of Tableau workbooks (.twb / .twbx) into fully deployable Power BI assets. It handles data connections, calculated field translation, visual layout mapping, PBIX file assembly, and direct publishing to the Power BI Service — all without opening Power BI Desktop.

Key Capabilities

- **75% automation coverage** — pattern-based DAX translation handles the majority of Tableau formulas without human input.
- **LLM fallback** — complex LOD expressions and table calculations are sent to Claude Sonnet or GPT-4o when pattern matching fails.
- **Touchless publish** — assembled .pbix files are uploaded directly to Power BI Service via REST API using a Service Principal.
- **XMLA/Fabric path** — for Premium and Fabric capacities, the semantic model (BIM) is deployed via the XMLA endpoint using Tabular Editor CLI.
- **Batch mode** — an entire folder of workbooks can be migrated and published in a single command.
- **Validation reporting** — every migration produces coverage metrics, a manual review checklist, and a full field lineage CSV.

Target Audience

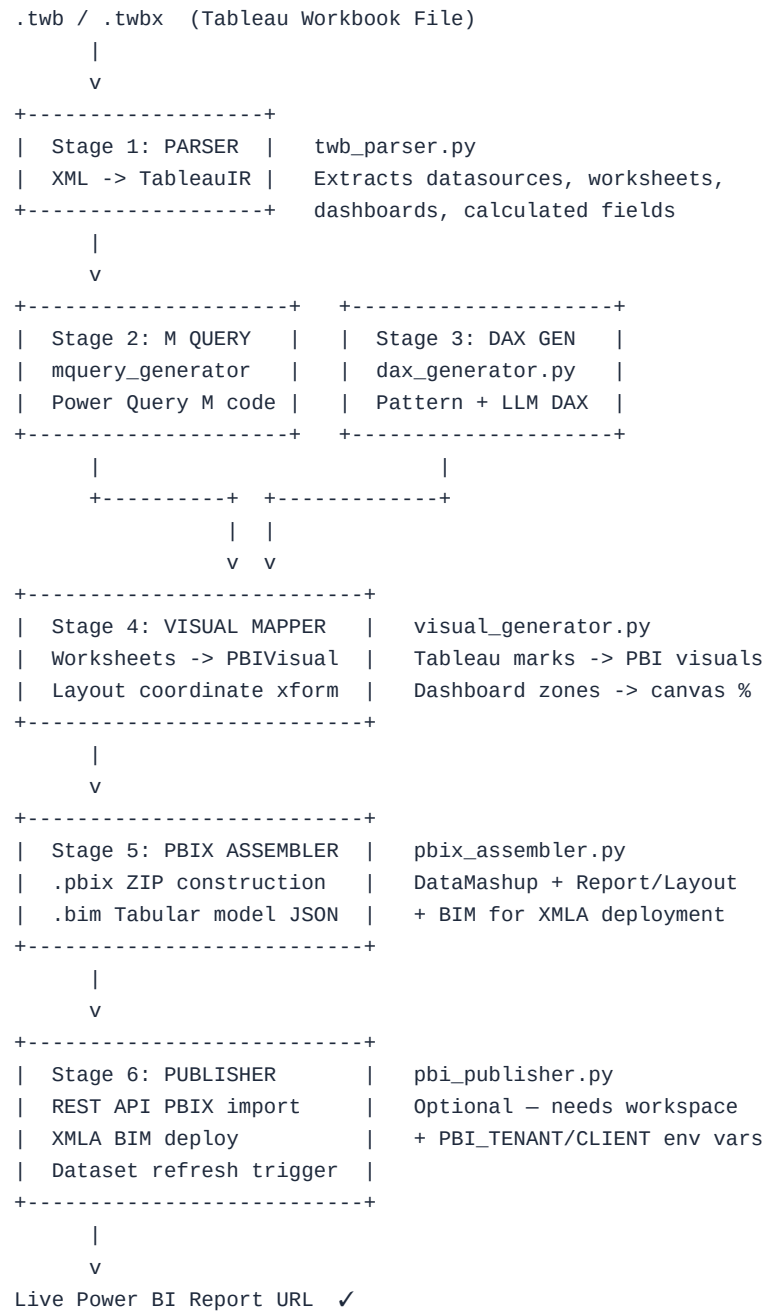
- **BI Managers** — understand automation coverage, review flagged items, oversee migration programmes.
- **Data Engineers** — extend the pattern catalog, wire up LLM keys, configure CI/CD pipelines.
- **Power BI Developers** — use generated DAX, M queries, and layout JSON as a starting scaffold.

Technology Stack

Component	Technology	Purpose
Parsing	defusedxml / ElementTree	Safe XML parsing of .twb files
Data Models	Pydantic v2	Typed IR and output schemas
DAX Translation	Regex + Claude Sonnet / GPT-4o	Pattern engine + LLM fallback
M Query Generation	String templates	Power Query source code
PBIX Assembly	zipfile (stdlib)	Builds deployable PBIX ZIP
BIM Generation	JSON	Tabular model for XMLA deployment
Publishing	Power BI REST API + MSAL	Touchless cloud deployment
CLI	Click	Command-line interface
Logging	Python logging	Structured progress output

2. Architecture Overview

6-Stage Pipeline



Module Dependency Map



```
■■■ generators/mquery_generator.py
■    ■■■ utils/models.py
■    ■■■ utils/mappings.py
■■■ generators/dax_generator.py
■    ■■■ utils/models.py
■    ■■■ utils/mappings.py
■■■ generators/visual_generator.py
■    ■■■ utils/models.py
■    ■■■ utils/mappings.py
■■■ generators/pbix_assembler.py
■    ■■■ utils/models.py
■■■ generators/pbi_publisher.py    (optional)
■■■ generators/report_writer.py
■    ■■■ utils/models.py
■    ■■■ validators/coverage_validator.py
■■■ validators/coverage_validator.py
    ■■■ utils/models.py
```

3. File Structure

```

tableau_to_pbi/
├──
├── __init__.py           Package marker
├── __main__.py           Enables: python3 -m tableau_to_pbi
├── cli.py                Click CLI – migrate / migrate-batch commands
├── pipeline.py           Orchestrates all 6 stages
├──
├── parsers/
│   ├── twb_parser.py     Parses .twb/.twbx XML -> TableauIR
│   └──
├── generators/
│   ├── mquery_generator.py Tableau datasources -> Power Query M
│   ├── dax_generator.py   Calculated fields -> DAX measures
│   ├── visual_generator.py Worksheets + dashboards -> PBI visuals
│   ├── pbix_assembler.py Assembles .pbix ZIP + .bim JSON
│   ├── pbi_publisher.py   REST API / XMLA publish to PBI Service
│   └── report_writer.py   Writes all output artefacts to disk
│   └──
├── validators/
│   ├── coverage_validator.py Field/visual/layout coverage checks
│   └──
├── utils/
│   ├── models.py          Pydantic data models (IR + PBI output)
│   ├── mappings.py        Static mapping tables (visuals, DAX, M)
│   └──
├── input_twb/             Drop .twb / .twbx files here
│   ├── sample_sales_dashboard.twb Bundled sample workbook
│   └──
├── output_pbi/            Generated artefacts (auto-created)
│   ├── {WorkbookName}/
│   │   ├── {name}.pbix     Deployable Power BI file
│   │   ├── {name}.bim      Tabular model (XMLA)
│   │   ├── powerquery/*.pq M query files
│   │   ├── dax/measures.dax DAX measures
│   │   ├── layout/report_pages.json
│   │   ├── ir/{name}.ir.json Intermediate Representation
│   │   ├── migration_report/
│   │   │   ├── summary.md
│   │   │   ├── manual_review.md
│   │   │   └── field_mapping.csv

```

4. Data Models — utils/models.py

All data structures are defined as **Pydantic v2 BaseModel** classes. This gives automatic validation, type coercion, and JSON serialisation (used for the IR output and layout files). There are two groups: Tableau IR models (what the parser produces) and Power BI output models (what the generators produce).

Tableau IR Models

TableauIR — Root object

Field	Type	Description
workbook_name	str	Stem of the source file (no extension)
source_file	str	Absolute path to the .twb/.twbx
datasources	list[TableauDataSource]	All data connections in the workbook
worksheets	list[TableauWorksheet]	All individual chart sheets
dashboards	list[TableauDashboard]	Dashboard layout definitions
parameters	list[TableauParameter]	Workbook-level parameters

TableauDataSource

Field	Type	Description
name	str	Internal Tableau name (used for field references)
caption	str	Display name shown in Tableau UI
connection_type	str	sqlserver / mysql / postgres / csv / excel etc.
server	str	Hostname extracted from connection XML
database	str	Database name
schema_name	str	Schema (e.g. dbo)
table	str	Base table name (empty if custom SQL)
custom_sql	str	Full SQL text when relation type = text
columns	list[TableauColumn]	All fields including calculated fields
joins	list[TableauJoin]	Join definitions between tables

TableauColumn

Field	Type	Description
name	str	Internal name (brackets stripped)
caption	str	Display label

Field	Type	Description
datatype	str	string / integer / real / date / datetime / boolean
role	str	dimension or measure
type	str	nominal / ordinal / quantitative / temporal
formula	str	Tableau expression (empty for raw fields)
is_calculated	bool	True if formula is non-empty
hidden	bool	Whether hidden in Tableau UI

TableauWorksheet

Field	Type	Description
name	str	Sheet tab name
datasource	str	Datasource name this sheet draws from
mark_type	str	bar / line / circle / text / pie / map / gantt etc.
encodings	list[TableauEncoding]	Shelf assignments (rows, cols, color, size ...)
filters	list[TableauFilter]	Filters applied to this worksheet
rows / cols	list[str]	Field names on Rows and Columns shelves

TableauEncoding

Represents a single shelf assignment in Tableau's Marks card.

Field	Type	Description
shelf	str	rows / cols / color / size / label / detail / tooltip / filter
field	str	Field name (brackets stripped)
aggregation	str	SUM / AVG / COUNT / COUNTD / MIN / MAX / ATTR / NONE
alias	str	Optional display alias

Power BI Output Models

Model	Key Fields	Purpose
MQueryTable	table_name, m_expression, connection_type	One Power Query M file per datasource
DAXMeasure	name, expression, confidence, needs_review	One DAX measure (calculated or implicit)
PBIVisual	visual_type, x/y/w/h_pct, fields, filters	One visual on a report page
PBIReportPage	name, visuals, source_dashboard	One Power BI report page
PBIOutput	m_queries, dax_measures, report_pages	Root container for all PBI artefacts

5. Stage 1 — Parser Agent — parsers/twb_parser.py

The parser is the entry point for every migration. It reads a Tableau workbook and produces a fully validated **TableauIR** object that all downstream stages consume.

File Format Handling

Format	Extension	How it is read
.twb	Plain XML	Read as UTF-8 text, parse with ElementTree
.twbx	ZIP archive	Open with zipfile, find the inner .twb, decode UTF-8

Security — XXE Prevention

The parser imports **defusedxml** instead of the standard library's ElementTree. This prevents XML External Entity (XXE) attacks — an attacker could craft a .twb file that reads local files (e.g. /etc/passwd) through XML entity expansion. defusedxml disables external entity resolution. If defusedxml is not installed, the parser falls back gracefully with a logged warning.

```
try:
    from defusedxml import ElementTree as ET # XXE-safe
except ImportError:
    from xml.etree import ElementTree as ET # fallback with warning
```

Helper Functions

`_attr(elem, *keys, default="")`

Tries each key in order and returns the first non-empty attribute value. Used throughout to handle Tableau's inconsistent attribute naming (e.g. 'server' vs 'servername').

`_norm_name(name)`

Strips Tableau's internal bracket notation. Examples:

```
_norm_name("[Sales]") -> "Sales"
_norm_name("[Orders].[Order ID]") -> "Order ID"
_norm_name("[Customer Name]") -> "Customer Name"
```

BUG FIX — `_split_shelf()` — Depth-aware Comma Split

Tableau shelf text like **SUM([Sales]),DATEADD('month',-1,[Order Date])** contains commas *inside* function arguments. The original code used `text.split(',')` which would incorrectly split DATEADD into three tokens. The fix tracks parenthesis depth:

```
def _split_shelf(text: str) -> list[str]:
    parts, depth, buf = [], 0, []
    for ch in text:
```

```
if ch == "(":
    depth += 1; buf.append(ch)
elif ch == ")":
    depth -= 1; buf.append(ch)
elif ch == "," and depth == 0: # only split at top level
    parts.append("".join(buf).strip()); buf = []
else:
    buf.append(ch)
if buf: parts.append("".join(buf).strip())
return [p for p in parts if p]
```

BUG FIX — `_get_field_name()` — Nested Expression Unwrapping

The original single-unwrap logic broke on nested aggregations like `SUM(YEAR([Order Date]))`. The fix iteratively strips outer function wrappers until a `[FieldName]` bracket is reached:

```
def _get_field_name(field_ref: str) -> str:
    inner = field_ref.strip()
    for _ in range(5): # max 5 nesting levels
        m = re.match(r"^[A-Za-z_]+\s*\((.+)\)$", inner)
        if not m: break
        inner = m.group(1).strip()
        if inner.startswith("["): break # reached the field name
    return _norm_name(inner)
```

Parsing Flow

Function	Input	Output
<code>parse_twb()</code>	Path to .twb/.twbx	TableauIR (root object)
<code>_parse_datasource()</code>	datasource XML element	TableauDataSource
<code>_parse_join()</code>	relation XML element	list[TableauJoin]
<code>_parse_worksheet()</code>	worksheet XML element	TableauWorksheet
<code>_parse_dashboard()</code>	dashboard XML element	TableauDashboard
<code>_parse_parameters()</code>	root XML element	list[TableauParameter]

6. Stage 2 — Power Query M Generator — generators/mquery_generator.py

Converts each TableauDataSource into a complete Power Query M expression that can be pasted directly into Power BI Desktop's Advanced Editor or embedded in the DataMashup of a .pbix file.

Connector Mapping — CONN_TYPE_MAP

Tableau Type	M Source Function	PBI Mode
sqlserver	Sql.Database("{server}", "{database}")	DirectQuery
mysql	MySQL.Database("{server}", "{database}")	DirectQuery
postgres	PostgreSQL.Database("{server}", "{database}")	DirectQuery
oracle	Oracle.Database("{server}")	DirectQuery
bigquery	GoogleBigQuery.Database()	Import
snowflake	Snowflake.Databases("{server}")	DirectQuery
redshift	AmazonRedshift.Database("{server}", "{database}")	Import
csv	Csv.Document(File.Contents("{table}"))	Import
excel	Excel.Workbook(File.Contents("{table}"), null, true)	Import
unknown	/* TODO: configure data source */	Import

Custom SQL Handling

When a Tableau datasource uses Custom SQL (relation type='text'), the generator emits a **Value.NativeQuery** call with query folding enabled:

```
let
    Source = Sql.Database("server", "SalesDB"),
    Data = Value.NativeQuery(Source,
        "SELECT * FROM dbo.Orders WHERE active = 1",
        null, [EnableFolding=true])
in
    Data
```

BUG FIX — Missing ExpandTableColumn

Power BI's **Table.NestedJoin** does NOT expose joined columns directly. It creates a new column containing a nested Table object. Without a subsequent **Table.ExpandTableColumn** step, users see an unexpanded [Table] column instead of the joined fields. The fix adds the expand step automatically:

```
# Step 1: Join
Joined_1 = Table.NestedJoin(
    PreviousStep, {"customer_id"},
    Customers, {"id"},
```

```
    "Customers_data", JoinKind.LeftOuter
),

# Step 2: Expand (the missing step, now added)
Expanded_1 = Table.ExpandTableColumn(
    Joined_1, "Customers_data",
    Table.ColumnNames(Customers),
    List.Transform(Table.ColumnNames(Customers), each "Customers." & _)
),
```

The prefix `Customers.` is added to each expanded column name to avoid collisions when multiple tables have identically named columns.

7. Stage 3 — DAX Generator — generators/dax_generator.py

The DAX generator is the most complex stage. It translates Tableau's expression language into DAX using a two-tier strategy: a fast regex pattern engine handles the majority of formulas; an LLM call handles anything the patterns cannot match.

Two-Tier Translation Strategy

```
Tableau Formula
|
v
[Tier 1] _apply_pattern() -- regex direct match
|
| No match
v
[Tier 1b] _translate_formula() recursive substitution loop
|         Replaces sub-expressions iteratively (max 10 passes)
|
| confidence == 0.0 AND use_llm == True
v
[Tier 2] _llm_translate() -- LLM call (Anthropic -> OpenAI cascade)
|
v
DAXMeasure(name, expression, confidence, needs_review)
```

FORMULA_PATTERNS Catalog — 40+ Patterns in 7 Categories

Category	Examples	Confidence
Basic aggregates	SUM, AVG, COUNT, COUNTD, MIN, MAX, MEDIAN	0.90–0.95
Null handling	ISNULL, ZN, IFNULL	0.85–0.90
String functions	UPPER, LOWER, TRIM, LEN, LEFT, RIGHT, MID, CONTAINS, STARTSWITH, ENDSWITH, REPLACE	0.80–0.95
Date functions	TODAY, NOW, YEAR, MONTH, DAY, DATETRUNC, DATEADD, DATEDIFF	0.80–0.95
LOD expressions	FIXED, INCLUDE, EXCLUDE	0.55–0.65
Table calculations	RUNNING_SUM, WINDOW_SUM, RANK, RUNNING_AVG	0.45–0.55
Conditional	IF/THEN/ELSE/END, IIF, ATTR	0.75–0.85

Pattern Tuple Structure

Each entry in FORMULA_PATTERNS is a 4-tuple:

```
(
    r"SUM\(\[.(+?)\]\)",      # regex pattern (capture groups map to {field})
```

```
"SUM({table}[{field}]]", # DAX template with {placeholder} substitution
0.95, # confidence score (0.0 - 1.0)
"" # review note (empty = no review needed)
)
```

BUG FIX — Python Closure over Loop Variable

The recursive substitution loop calls `re.sub(pattern, _replacer, ...)` for each pattern. The `_replacer` function was defined inside the for loop. In Python, closures capture variables by *reference*, not by value. By the time `re.sub()` calls `_replacer`, the loop variable `template` has already advanced to the *last* pattern in the list — every substitution used the wrong template silently.

```
# BEFORE (bug): closure captures template by reference
for pattern, template, confidence, note in FORMULA_PATTERNS:
    def _replacer(m):
        dax = template # WRONG: always the last template

# AFTER (fix): default arguments capture values at definition time
for pattern, template, confidence, note in FORMULA_PATTERNS:
    def _replacer(m, _tmpl=template, _conf=confidence, _note=note):
        dax = _tmpl # CORRECT: captured at loop iteration
```

LOD Expression Translation

Tableau's Level of Detail expressions are the hardest constructs to translate. They evaluate at a different grain than the current visual filter context.

Tableau LOD	Generated DAX	Confidence
{FIXED [Region] : SUM([Sales])}	CALCULATE(SUM(T[Sales]), REMOVEFILTERS(), VALUES(T[Region]))	0.65
{INCLUDE [Product] : AVG([Price])}	CALCULATE(AVERAGE(T[Price]), SUMMARIZE(T, T[Product]))	0.55
{EXCLUDE [Month] : SUM([Sales])}	CALCULATE(SUM(T[Sales]), REMOVEFILTERS(T[Month]))	0.60

Note: All LOD expressions are flagged for human review regardless of confidence score because DAX filter context semantics differ fundamentally from Tableau's LOD evaluation order.

LLM Fallback — `_llm_translate()`

When pattern matching yields confidence 0.0 AND `use_llm=True`, the formula is sent to an LLM with a structured prompt. The cascade tries Anthropic first (Claude Sonnet — matches the project stack), then falls back to OpenAI (GPT-4o), then returns a TODO comment if neither key is configured.

```
prompt = f"""
Convert this Tableau formula to a DAX measure expression.
Table name in the data model: {table}
Tableau formula:
{formula}

Return ONLY the DAX expression, no explanation.
"""
```

8. Stage 4 — Visual Generator — generators/visual_generator.py

Maps Tableau worksheets and dashboard zone positions into Power BI PBIVisual and PBIReportPage objects.

Visual Type Mapping — MARK_TO_VISUAL

Tableau Mark	Power BI Visual	Confidence	Notes
bar	barChart	0.95	
bar_h	barChart	0.90	Set orientation=horizontal
line	lineChart	0.95	
area	areaChart	0.90	
circle	scatterChart	0.85	
square	scatterChart	0.80	Map shape to scatter
text	tableEx	0.85	
pie	pieChart	0.90	
map / filled_map	filledMap	0.80	Verify geo field
symbol_map	map	0.75	Verify lat/lon fields
gantt	gantt	0.55	Requires AppSource visual
polygon	shapeMap	0.50	Requires custom shape file
density	map	0.45	Density layer not native
automatic	barChart	0.70	Tableau chose automatic — review

Coordinate Normalisation

Tableau stores zone positions in absolute pixels relative to the dashboard canvas. Power BI uses percentage-based positioning relative to the report canvas (1280 × 720 pts). The conversion is straightforward:

```
# Tableau dashboard is e.g. 1366 x 768 px
# Zone at x=683, y=0, w=683, h=384

x_pct = round((683 / 1366) * 100, 2)  # = 50.0%
y_pct = round((0 / 768) * 100, 2)    # = 0.0%
w_pct = round((683 / 1366) * 100, 2)  # = 50.0%
h_pct = round((384 / 768) * 100, 2)   # = 50.0%
```

Field Well Mapping — SHELF_TO_WELL

Tableau Shelf	Power BI Field Well
rows	values
cols	axis
color	legend
size	size
label	labels
detail	details
tooltip	tooltips
filter	filters

Orphan Worksheet Handling

Worksheets that are not placed on any dashboard are still migrated — each becomes its own standalone report page with default 50×50% visual placement and is flagged in the migration report for manual positioning.

9. Stage 5 — PBIX Assembler — generators/pbix_assembler.py

This stage eliminates the need for Power BI Desktop entirely. It constructs a valid .pbix ZIP file from the generated M queries and report layout JSON.

PBIX File Format

A .pbix file is a **renamed ZIP archive** with a specific set of mandatory files:

File inside ZIP	Content	Built by
[Content_Types].xml	MIME type manifest	Hard-coded template
Version	Format version string (4.21)	Hard-coded constant
SecurityBindings	Empty placeholder	Empty string
DataMashup	ZIP-in-ZIP containing Power Query M	<code>_build_mashup_zip()</code>
Report/Layout	JSON: pages, visuals, positions	<code>_build_layout()</code>

DataMashup — ZIP inside the PBIX

The DataMashup entry is itself a ZIP archive containing:

```
DataMashup (binary ZIP)
  ■■■ [Content_Types].xml      MIME types for .m files
  ■■■ Section1.m              Combined Power Query M section document
  ■■■ Formulas/Section1/      Legacy path (older PBI Service versions)
  ■ ■■■ Package/Formulas/
  ■ ■■■ Section1.m            Same content – duplicate for compatibility
  ■■■ Config/Package.json      Minimal metadata {"version": "2.0"}
```

The Section1.m file uses Tableau section syntax with each datasource as a named shared query:

```
section Section1;

shared #"Orders" =
  let
    Source = Sql.Database("sql-prod.company.com", "SalesDB"),
    Table = Source[[Schema="dbo", Item="Orders"]][Data]
  in
    Table;

shared #"Customers" =
  let
    Source = Sql.Database("sql-prod.company.com", "SalesDB"),
    Table = Source[[Schema="dbo", Item="Customers"]][Data]
  in
    Table;
```

Report/Layout JSON

The layout follows Power BI's internal schema:

```
{
  "id": 0,
  "sections": [
    {
      "id": 0,
      "name": "page0001",
      "displayName": "Executive Dashboard",
      "visualContainers": [
        {
          "x": 0, "y": 0, "z": 0,
          "width": 640, "height": 360,
          "config": "{ ... stringified visual JSON ... }",
          "filters": "[]"
        }
      ]
    }
  ]
}
```

Note: The config field is a JSON string (stringified JSON, not a nested object). This is a quirk of the Power BI internal format — the outer Layout JSON contains JSON-encoded strings for per-visual configuration.

BIM — Tabular Model for XMLA Deployment

build_bim() generates a JSON file in the Tabular Model BIM format, which is the format understood by Tabular Editor CLI and the XMLA endpoint. It includes:

- A hidden **_Measures** table containing all DAX measures with their source formula, confidence score, and review flag as annotations.
- One table entry per datasource, with partition mode set to import or directQuery based on the CONN_TYPE_MAP setting.
- Compatibility level 1567 (Power BI Premium / Fabric).

10. Stage 6 — Publisher — generators/pbi_publisher.py

The publisher makes the migration completely touchless. When a workspace is specified, it authenticates, uploads the PBIX, waits for import completion, and triggers a dataset refresh — returning a live report URL.

Authentication — Two Paths

Path	When Used	Env Vars Required	Best For
Service Principal (client credentials)	PBI_CLIENT_SECRET is set	PBI_TENANT_ID PBI_CLIENT_ID PBI_CLIENT_SECRET	CI/CD pipelines, automated batch runs
Device Flow (interactive)	No CLIENT_SECRET	PBI_TENANT_ID PBI_CLIENT_ID	First-time setup, developer testing

Service Principal Flow

POST https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/token

Body:

```
grant_type    = client_credentials
client_id     = {PBI_CLIENT_ID}
client_secret = {PBI_CLIENT_SECRET}
scope         = https://analysis.windows.net/powerbi/api/.default
```

Device Flow

When no client secret is set, the user sees:

Device-flow auth required:

```
→ Visit:  https://microsoft.com/devicelogin
→ Code:   ABC123
```

The MSAL library polls until the user completes browser authentication.

REST API — PBIX Import

POST [/v1.0/myorg/groups/{workspace_id}/imports](#)
[?datasetDisplayName={name}&nameConflict=Overwrite](#)

Content-Type: multipart/form-data

Body: (binary PBIX file)

Response 202:

```
{ "id": "import-job-guid" }
```

Polling — wait_for_import()

Polls every 5 seconds up to 300 seconds:

```
GET /v1.0/myorg/groups/{workspace_id}/imports/{import_id}
```

Response when complete:

```
{
  "importState": "Succeeded",
  "datasets": [{ "id": "...", "name": "..."}],
  "reports": [{ "id": "...", "name": "..."}]
}
```

XMLA / Tabular Editor CLI

For Premium and Fabric workspaces, the BIM file is deployed via Tabular Editor CLI:

```
TabularEditor model.bim \
  -D "powerbi://api.powerbi.com/v1.0/myorg/MyWorkspace" \
  MyDatasetName \
  -O # Overwrite existing
  -S # Deploy schema changes
  -P # Deploy partition changes
```

Full publish() Orchestration

Step	Action	Function
1	Acquire OAuth2 token	get_token()
2	Resolve workspace name → GUID	resolve_workspace_id()
3a	Deploy BIM via XMLA (if --xmla set)	deploy_bim_via_xmla()
3b	Upload PBIX via REST API	import_pbix()
4	Poll until import complete	wait_for_import()
5	Trigger dataset refresh	trigger_refresh()
6	Return live report URL	Composed from report_id

11. Validator — validators/coverage_validator.py

The validator audits the generated PBIOutput against the source TableauIR and produces a structured ValidationReport used to generate the summary.md and manual_review.md files.

ValidationIssue

Field	Values	Description
severity	ERROR / WARNING / INFO	ERROR = must fix; WARNING = should fix; INFO = advisory
category	datasource / field / dax / visual / layout	Which part of the migration this affects
item	str	Name of the specific element (measure name, worksheet name etc.)
message	str	Human-readable description of the issue

Coverage Metrics

Metric	Formula	Weight
Field Coverage %	translated_fields / total_calculated_fields × 100	40%
Visual Coverage %	visuals_auto / total_visuals × 100	60%
Overall Coverage %	field_coverage × 0.4 + visual_coverage × 0.6	—

BUG FIX — visuals_auto Counter

The original code only incremented visuals_auto for visuals that had *no* review notes — meaning a bar chart with a minor note (e.g. 'verify geo field') was counted as zero, deflating the visual coverage percentage. The fix separates 'mapped' from 'needs review':

```
# BEFORE (bug): flagged visuals excluded from visuals_auto
for vis in all_visuals:
    if vis.needs_review:
        report.visuals_flagged += 1
    else:
        report.visuals_auto += 1    # WRONG: only clean visuals counted

# AFTER (fix): all mapped visuals count toward coverage
for vis in all_visuals:
    report.visuals_auto += 1        # always – represents "mapped"
    if vis.needs_review:
        report.visuals_flagged += 1 # sub-count: mapped but flagged
```

Five Validation Checks

Check	What It Verifies	Severity if Failed
Datasource coverage	Every IR datasource has a generated M query	ERROR

Check	What It Verifies	Severity if Failed
Calculated field coverage	Every calculated field has a DAX measure	WARNING
DAX review flags	Measures with confidence < 0.70 are flagged	WARNING
DAX zero-confidence	Measures that could not be translated at all	ERROR
Visual coverage	Every visual has a mapped type	WARNING
Layout coverage	Every worksheet placed on a report page	WARNING
Parameter detection	Tableau parameters need manual What-If setup	INFO

12. Report Writer — generators/report_writer.py

Writes all generated artefacts to a structured output folder. Every file is UTF-8 encoded. The writer never overwrites — running the pipeline twice will overwrite previous outputs silently (directories are created with `exist_ok=True`).

Output Files

File	Content	Primary Consumer
{name}.ir.json	Full TableauIR as Pydantic JSON	Debugging / re-run
{name}.pbix	Deployable Power BI file	Power BI Service / Desktop
{name}.bim	Tabular model JSON	Tabular Editor / XMLA endpoint
powerquery/*.pq	One M query per datasource	Power BI Advanced Editor
dax/measures.dax	All DAX measures with [REVIEW] tags	DAX Editor / developer
layout/report_pages.json	Visual specs with % coordinates	Developer reference
migration_report/summary.md	Coverage % + source stats + next steps	BI Manager
migration_report/manual_review.md	Errors → Warnings → Info items	Developer
migration_report/field_mapping.csv	10-column Tableau→DAX lineage	Stakeholder sign-off

field_mapping.csv — 10-Column Lineage Table

Column	Description
Source Table	Datasource caption or name
Tableau Field	Field caption or name
Type	Pydantic datatype (string / real / date etc.)
Is Calculated	True / False
Tableau Formula	Original expression (blank for raw fields)
DAX Measure	Generated DAX measure name
DAX Expression	First 200 chars of generated DAX
Confidence	Translation confidence as percentage
Needs Review	True / False
Review Reason	Human-readable reason for review flag

BUG FIX — _safe() Filename Sanitisation

The original sanitiser could produce empty strings or all-underscore names for datasources with non-ASCII display names (e.g. Japanese characters). This caused a `FileNotFoundError` when attempting to write the .pq file:

```
# BEFORE (bug): could return "" or "___"
def _safe(name):
    return "".join(c if c.isalnum() or c in "-_" else "_" for c in name)

# AFTER (fix): strip leading/trailing underscores, fallback to "unnamed"
def _safe(name):
    result = "".join(c if c.isalnum() or c in "-_" else "_" for c in name).strip("_")
    return result or "unnamed"
```

13. Pipeline Orchestrator — pipeline.py

The pipeline module wires all stages together into a single `migrate_workbook()` function and provides a batch wrapper.

migrate_workbook() — Data Flow

Stage	Function Called	Input	Output Added to PBIOutput
1 — Parse	<code>parse_twb()</code>	<code>twb_path</code>	TableauIR
2 — M Query	<code>generate_m_queries()</code>	<code>ir.datasources</code>	<code>m_queries</code>
3 — DAX	<code>generate_dax_measures()</code>	<code>ir (full)</code>	<code>dax_measures</code>
4 — Visuals	<code>generate_report_pages()</code>	<code>ir (full)</code>	<code>report_pages</code>
5 — Assemble	<code>validate() + write_outputs()</code> <code>+ assemble_pbix() + build_bim()</code>	<code>ir + PBIOutput</code>	<code>.pbix + .bim + report files</code>
6 — Publish	<code>publish() (optional)</code>	<code>pbix_path + bim</code>	<code>report_url</code> in summary

Function Signature

```
def migrate_workbook(  
    twb_path:      Path,          # source .twb or .twbx  
    output_dir:    Path,          # base output directory  
    use_llm:       bool = True,   # enable LLM fallback for DAX  
    workspace:     str  = None,    # PBI workspace name/GUID (enables publish)  
    xmla_endpoint: str  = None,    # XMLA URL (Premium/Fabric only)  
    overwrite:     bool = True,    # overwrite existing PBI report  
) -> dict:                       # summary dict with coverage + URL
```

Error Isolation in migrate_batch()

Each workbook is wrapped in a try/except block. A failure in one workbook logs the error and adds it to the summary dict with an 'error' key, but processing continues for all remaining files in the batch.

14. CLI — cli.py

Built on **Click**. Two commands — migrate (single file) and migrate-batch (folder) — share publish options via a DRY decorator pattern.

migrate Command

Option	Type	Default	Description
--input	Path (required)	—	.twb or .twbx file to migrate
--output	Path	output_pbi/	Base output directory
--no-llm	Flag	False	Disable LLM fallback for DAX
--workspace	str	None	PBI workspace — enables publish
--xmla	str	None	XMLA endpoint URL for Fabric/Premium
--no-overwrite	Flag	False	Abort if report already exists
--verbose / -v	Flag	False	Show DEBUG-level log output

_add_publish_options — DRY Decorator Pattern

Rather than duplicating the three publish options across both commands, they are defined in a list and applied via a decorator function:

```
_publish_options = [
    click.option("--workspace", ...),
    click.option("--xmla", ...),
    click.option("--no-overwrite", ...),
]

def _add_publish_options(cmd):
    for opt in reversed(_publish_options): # reversed: Click applies in reverse order
        cmd = opt(cmd)
    return cmd

@cli.command("migrate")
@_add_publish_options # injects all three options
def migrate_cmd(...):
    ...
```

_print_summary() — Colour-coded Output

Condition	Colour	Example
errors == 0	Green	✓ sales_dashboard coverage=100% errors=0 warnings=2
errors > 0	Yellow	✓ complex_report coverage=78% errors=3 warnings=5
Migration failed	Red	✗ broken_file: ValueError: No .twb found inside

Condition	Colour	Example
Live URL printed	Cyan	Live → https://app.powerbi.com/groups/.../reports/...

15. Mapping Tables — utils/mappings.py

All mapping tables are pure Python dictionaries and lists — no external config files. This makes them easy to version-control, diff, and extend without any YAML parsing overhead.

AGG_MAP — Tableau → DAX Aggregations

Tableau	DAX Function
SUM	SUM
AVG	AVERAGE
COUNT	COUNT
COUNTD	DISTINCTCOUNT
MIN	MIN
MAX	MAX
MEDIAN	MEDIAN
STDEV	STDEV.S
STDEVP	STDEV.P
VAR	VAR.S
VARP	VAR.P
ATTR	SELECTEDVALUE
YEAR	YEAR
QUARTER	QUARTER
MONTH	MONTH
DAY	DAY

DTYPE_MAP — Data Type Mapping

Tableau Type	Power BI Type
string	Text
integer	Whole Number
real	Decimal Number
boolean	True/False
date	Date
datetime	Date/Time
spatial	Text (no direct PBI equivalent)

Extending the Pattern Catalog

To add a new Tableau function pattern, append a 4-tuple to FORMULA_PATTERNS in mappings.py:

```
(
    r"SPACE\((\d+)\)",          # regex for SPACE(n)
    "REPT(" ", {field})",      # DAX template ({field} = capture group 1)
    0.90,                       # confidence
    ""                          # no review note
)
```

16. Bug Fixes — Code Review Findings

Six bugs were identified in a structured code review and fixed before this document was written. All fixes are in the current codebase.

ID	Severity	File	Root Cause	Fix Applied
B1	CRITICAL	dax_generator.py	Python closure captured loop variable template instead of capturing all references to it. This substitution placed the first	Defaulting capture all references to it. This substitution placed the first
B2	CRITICAL	mquery_generator.py	Table.NestedJoin creates a nested Table column. ExportTableColumn after joining NestedJoin with	Added MixJoinTableColumn after joining NestedJoin with
B3	CRITICAL	twb_parser.py	Shelf text split on all commas. DATEADD(Replace(Date)) was with respect to shift() into date, the case splitting	Replace(Date) was with respect to shift() into date, the case splitting
B4	SECURITY	twb_parser.py	xml.etree.ElementTree is not XXE-safe. A malicious XML document could load local files, full disk and Memory, log	Switched to lxml to use lxml.etree.ElementTree and lxml.etree.ElementTree
B5	SAFETY	report_writer.py	_safe() could return empty string or all-underscores. Added strip() as SQL database fallbacks to cause() FileNotFound	Added strip() as SQL database fallbacks to cause() FileNotFound
B6	CORRECTNESS	Coverage_validator.py	visuals_auto counter excluded flagged visuals, making visual coverage % for initially loaded visuals, visuals safe	visuals_auto counter excluded flagged visuals, making visual coverage % for initially loaded visuals, visuals safe

17. Enhancement Roadmap

Fourteen enhancements identified in the code review, prioritised by value.

High Value — Do First

ID	Enhancement	Effort	Impact
E1	ELSEIF chain → SWITCH(TRUE(), ...) Multi-branch Tableau IFs are very common	Low	High
E2	Auto-generate a standard M Date table Required for RUNNING_SUM / DATESYTD	Medium	High
E3	Named Groups → DAX SWITCH skeleton Tableau Named Groups are common in BI workbooks	Medium	High
E4	Multi-datasource blending warning PBI needs model relationships, not view-level blends	Low	High
E5	Multi-dimension FIXED LOD pattern {FIXED [A],[B] : ...} not matched by current patterns	Low	High

Medium Value — Next Sprint

ID	Enhancement	Effort	Impact
E6	Tableau Set → DAX calculated table CALCULATETABLE(VALUEs(...), ...) pattern	Medium	Medium
E7	Bin → DAX calculated column FLOOR([Field]/n)*n with bin size from IR	Low	Medium
E8	Conditional formatting spec in layout JSON Highlight table background rules → PBI cond. format	Medium	Medium
E9	Parameter → What-If M snippet Auto-generate the slider M query + slicer visual spec	Low	Medium
E10	--diff mode: only output changed measures/visuals Re-run on updated .twb without full regeneration	Medium	Medium

Testing & Quality

ID	Enhancement	Effort	Impact
E11	Unit tests for _translate_formula() Table-driven tests for every FORMULA_PATTERNS entry	Low	Medium
E12	Golden output comparison Store expected .dax / .pq and fail CI on regression	Medium	High

ID	Enhancement	Effort	Impact
E13	LLM response validation Strip markdown fences, verify DAX syntax before accepting	Low	Medium
E14	batch_summary.json output Machine-readable summary for downstream tooling	Low	Low

18. End-to-End Example Walkthrough

This section traces the bundled `sample_sales_dashboard.twb` through all 6 pipeline stages.

Source Workbook — What the Parser Extracts

Element	Count	Details
Datasources	1	orders_ds — SQL Server, dbo.Orders, DirectQuery
Worksheets	6	Sales by Region, Profit by Category, Sales Trend, Customer Scatter, Sales Map, KPI Summary
Dashboards	2	Executive Dashboard (4 visuals), Customer Analysis (2 visuals)
Calculated Fields	7	Profit Ratio, Sales per Customer, Days to Ship, Above Average Sales, Profit Band, YTD Sales, Customer Upper
Parameters	0	None in this workbook

Stage 3 — DAX Generated for Each Calculated Field

Tableau Formula	Generated DAX	Confidence
SUM([Profit])/SUM([Sales])	SUM(Orders[Profit]) / SUM(Orders[Sales])	0.95
SUM([Sales])/COUNTD([Customer ID])	SUM(Orders[Sales]) / DISTINCTCOUNT(Orders[Customer ID])	0.95
DATEDIFF('day',[Order Date],[Ship Date])	DATEDIFF(Orders[Order Date], Orders[Ship Date], DAY)	0.80
SUM([Sales]) > {FIXED [Region] : AVG(SUM([Sales]))}	SUM(Orders[Sales]) > CALCULATE(AVERAGE(...), REMOVEFILTERS(), VALUES(...))	0.95
IF [Profit] > 500 THEN 'High' ELSE ... END	IF(Orders[Profit] > 500, "High", IF(..., "Medium", "Loss"))	0.80
RUNNING_SUM(SUM([Sales]))	CALCULATE(SUM(Orders[Sales]), DATESYTD(Date[Date]))	0.50
UPPER([Customer Name])	UPPER(Orders[Customer Name])	0.95

Stage 4 — Visual Mapping per Dashboard

Executive Dashboard (1366 × 768 px)

Worksheet	Tableau Mark	PBI Visual	Position (%)
Sales by Region	bar	barChart	x=0, y=0, w=50, h=50
Profit by Category	bar	barChart	x=50, y=0, w=50, h=50
Sales Trend	line	lineChart	x=0, y=50, w=50, h=50
KPI Summary	text	tableEx	x=50, y=50, w=50, h=50

Customer Analysis (1366 × 768 px)

Worksheet	Tableau Mark	PBI Visual	Position (%)
Customer Scatter	circle	scatterChart	x=0, y=0, w=66.7, h=100
Sales Map	filled_map	filledMap	x=66.7, y=0, w=33.3, h=100

Stage 5 — Output Files Written

```
output_pbi/sample_sales_dashboard/  
  sample_sales_dashboard.pbix      2.2 KB  (deployable)  
  sample_sales_dashboard.bim       3.1 KB  (XMLA)  
powerquery/  
  Orders.pq                        190 B  
dax/  
  measures.dax                     1.4 KB  (12 measures, 0 flagged)  
layout/  
  report_pages.json                5.9 KB  (2 pages, 6 visuals)  
ir/  
  sample_sales_dashboard.ir.json   4.2 KB  
migration_report/  
  summary.md                       Overall coverage: 100.0%  
  manual_review.md                 0 errors, 0 warnings  
  field_mapping.csv                17 rows (all fields)
```

19. Deployment Guide

Step 1 — Install Dependencies

```
pip3 install defusedxml click pydantic requests msal reportlab
# Optional: for LLM DAX translation
pip3 install anthropic openai
```

Step 2 — One-Time Azure App Registration

Step	Action
1	portal.azure.com → Azure Active Directory → App registrations → New registration
2	Name: tableau-pbi-migrator — note the Tenant ID and Client ID from Overview
3	Certificates & secrets → New client secret — copy the secret value immediately
4	API permissions → Add → Power BI Service → Delegated: Dataset.ReadWrite.All, Report.ReadWrite.All, Workspace.Read.All → Grant admin consent
5	In Power BI Service → Your Workspace → Access → add the app as Member or Admin

Step 3 — Environment Variables

```
# Add to ~/.zshrc or ~/.bash_profile

# Required for publish
export PBI_TENANT_ID="xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"
export PBI_CLIENT_ID="xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"
export PBI_CLIENT_SECRET="your-secret-value"

# Optional: LLM DAX translation
export ANTHROPIC_API_KEY="sk-ant-..." # Claude Sonnet
export OPENAI_API_KEY="sk-..." # GPT-4o (fallback)
```

Step 4 — Run Commands

```
# Local only — single file
python3 -m tableau_to_pbi migrate \
  --input tableau_to_pbi/input_twb/MyDashboard.twb

# Local only — batch
python3 -m tableau_to_pbi migrate-batch \
  --input-dir tableau_to_pbi/input_twb/

# Touchless publish — single file
python3 -m tableau_to_pbi migrate \
  --input tableau_to_pbi/input_twb/MyDashboard.twb \
  --workspace "Analytics Team"

# Touchless publish — batch
```

```
python3 -m tableau_to_pbi migrate-batch \  
  --input-dir tableau_to_pbi/input_twbl/ \  
  --workspace "Analytics Team"  
  
# Fabric / Premium – XMLA semantic model deploy  
python3 -m tableau_to_pbi migrate \  
  --input tableau_to_pbi/input_twbl/MyDashboard.twbl \  
  --workspace "Analytics Team" \  
  --xmla "powerbi://api.powerbi.com/v1.0/myorg/Analytics Team"  
  
# Pattern-only (no LLM, no API key needed)  
python3 -m tableau_to_pbi migrate-batch \  
  --input-dir tableau_to_pbi/input_twbl/ \  
  --no-llm  
  
# Verbose output (shows every visual and measure as it's generated)  
python3 -m tableau_to_pbi migrate \  
  --input tableau_to_pbi/input_twbl/MyDashboard.twbl -v
```

20. Glossary

Term	Definition
BIM	Business Intelligence Model. A JSON file in the Tabular Model format that defines tables, columns, measures, and relationships for a data model.
DAX	Data Analysis Expressions. The formula language used in Power BI, SSAS Tabular, and Excel Power Pivot to define calculated columns, measures, and other data model objects.
IR	Intermediate Representation. A structured, language-agnostic JSON object that captures the full meaning of a Tableau workbook. Acts as a bridge between Tableau and other data processing engines.
LOD	Level of Detail expression. A Tableau construct that allows calculations to be evaluated at a different grain than the current visual. Types include fixed, partition, and context.
M Query / Power Query	The functional query language used in Power BI to connect to data sources, transform data, and define tables. Stored as .m or .pq files.
PBIX	Power BI Desktop file format. A ZIP archive containing a DataMashup (Power Query M), Report/Layout (JSON), and optionally a Data Lake connection file.
TWB	Tableau Workbook. A plain XML file (extension .twb) containing all datasource definitions, calculated fields, worksheet configurations, and other metadata.
TWBX	Tableau Packaged Workbook. A ZIP archive (extension .twbx) containing a .twb file plus embedded data extracts (.hyper files).
XMLA	XML for Analysis. A SOAP-based protocol for querying and managing Analysis Services databases. Power BI Premium and Fabric use this for semantic model connectivity.
XXE	XML External Entity attack. A vulnerability where an XML parser processes external entity references in untrusted XML input, potentially leading to information disclosure or denial of service.

Generated by the Tableau → Power BI Migration Accelerator documentation tool. All code and documentation © LatentView Analytics 2026.